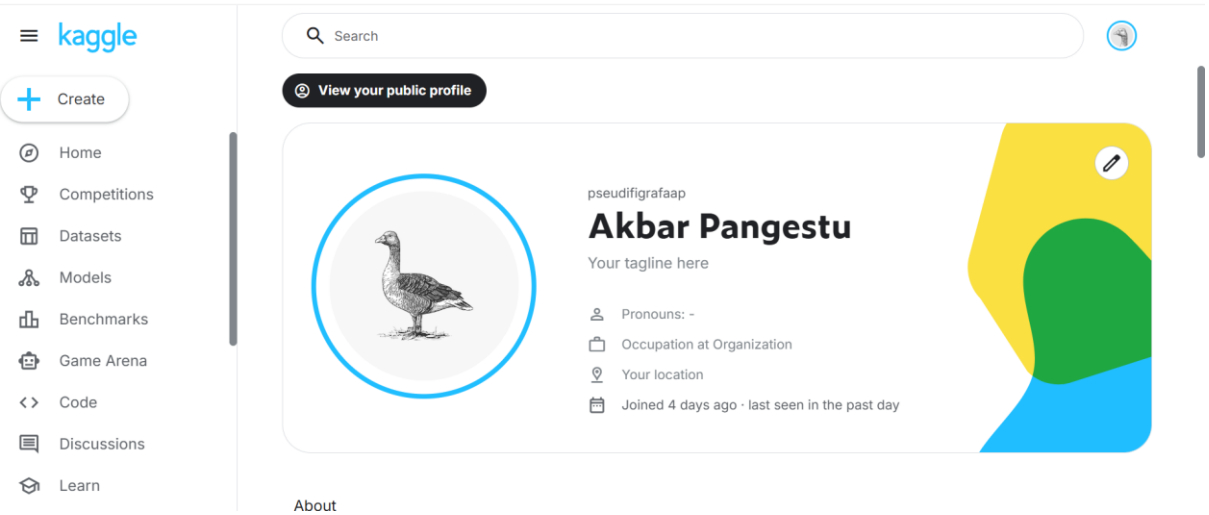
Laporan Tugas Big Data Analisis Dataset Soil Mosture

Disusun Oleh:
Akbar Pangestu(23082010238)

1.Set up and Acquisition

A.Daftar akun keggle



B.Mengunduh dataset via api jupyter notebook

a. Penginstalan libary yang dibutuhkan

```
[1]: !pip install opendatasets
     !pip install pandas
```

Penjelasan:

- !pip install opendatasets
  Perintah ini menginstal pustaka opendatasets yang memudahkan pengunduhan dataset dari berbagai sumber daring seperti Kaggle, Google Drive, dan lain-lain. Meskipun dalam kode selanjutnya pustaka ini tidak digunakan, instalasi tetap dilakukan sebagai langkah antisipasi jika diperlukan.

- !pip install pandas
  Perintah ini menginstal pustaka pandas, yang merupakan pustaka utama untuk manipulasi dan analisis data dalam Python. pandas akan digunakan untuk membaca dan mengolah file CSV setelah dataset berhasil diunduh dan diekstraksi.

b.Mendownload dataset

```
[2]: !kaggle datasets download -d amirmohammdjalili/soil-moisture-dataset

Dataset URL: https://www.kaggle.com/datasets/amirmohammdjalili/soil-moisture-dataset
License(s): unknown
Downloading soil-moisture-dataset.zip to C:\Users\user

  0%|          | 0.00/90.9k [00:00<?, ?B/s]
100%|##########| 90.9k/90.9k [00:00<00:00, 350kB/s]
100%|##########| 90.9k/90.9k [00:00<00:00, 349kB/s]
```

Penjelasan:

- !kaggle datasets download
  Perintah ini memanggil Kaggle API untuk mengunduh dataset. Agar berhasil, pengguna harus telah menginstal kaggle (biasanya melalui pip install kaggle) dan

mengonfigurasi kredensial API dengan meletakkan file kaggle.json di direktori yang sesuai (misalnya ~/.kaggle/ pada Linux/Mac
atau C:\Users\<nama_pengguna>\.kaggle\ pada Windows).

- -d amirmohammdjalili/soil-moisture-dataset
  Opsi -d menentukan dataset yang akan diunduh dengan format pemilik/nama-dataset. Dalam kasus ini, dataset yang diunduh adalah soil-moisture-dataset milik pengguna amirmohammdjalili.

c.Extrak zip file dataset

```
import zipfile
import os

zip_file_name = 'soil-moisture-dataset.zip'

with zipfile.ZipFile(zip_file_name, 'r') as zip_ref:
    zip_ref.extractall(r'C:\Users\user\.kaggle')
    print(f"File {zip_file_name} berhasil diekstrak.")

os.remove(zip_file_name)
print("File zip telah dihapus.")
```

Penjelasan:

- import zipfile
  Modul ini digunakan untuk bekerja dengan file ZIP, termasuk membaca dan mengekstrak isinya.
- import os
  Modul ini menyediakan fungsi untuk berinteraksi dengan sistem operasi, seperti menghapus file.
- zip_file_name = 'soil-moisture-dataset.zip'
  Variabel ini menyimpan nama file ZIP yang telah diunduh.
- with zipfile.ZipFile(zip_file_name, 'r') as zip_ref:
  Membuka file ZIP dalam mode baca ('r'). Penggunaan with memastikan file ditutup secara otomatis setelah selesai.
- zip_ref.extractall(r'C:\Users\user\.kaggle')
  Mengekstrak seluruh isi file ZIP ke direktori tujuan, yaitu C:\Users\user\.kaggle. Direktori ini umumnya digunakan untuk menyimpan file konfigurasi Kaggle. Perlu diperhatikan bahwa pemilihan direktori ini mungkin kurang umum untuk menyimpan dataset; sebaiknya disesuaikan dengan struktur proyek yang lebih terorganisir.
- print(f"File {zip_file_name} berhasil diekstrak.")
  Menampilkan pesan konfirmasi bahwa proses ekstraksi telah selesai.
- os.remove(zip_file_name)
  Menghapus file ZIP setelah berhasil diekstraksi untuk menghemat ruang penyimpanan.
- print("File zip telah dihapus.")
  Menampilkan konfirmasi bahwa file ZIP telah dihapus.

d.Meletakan file dataset sesuai direktori

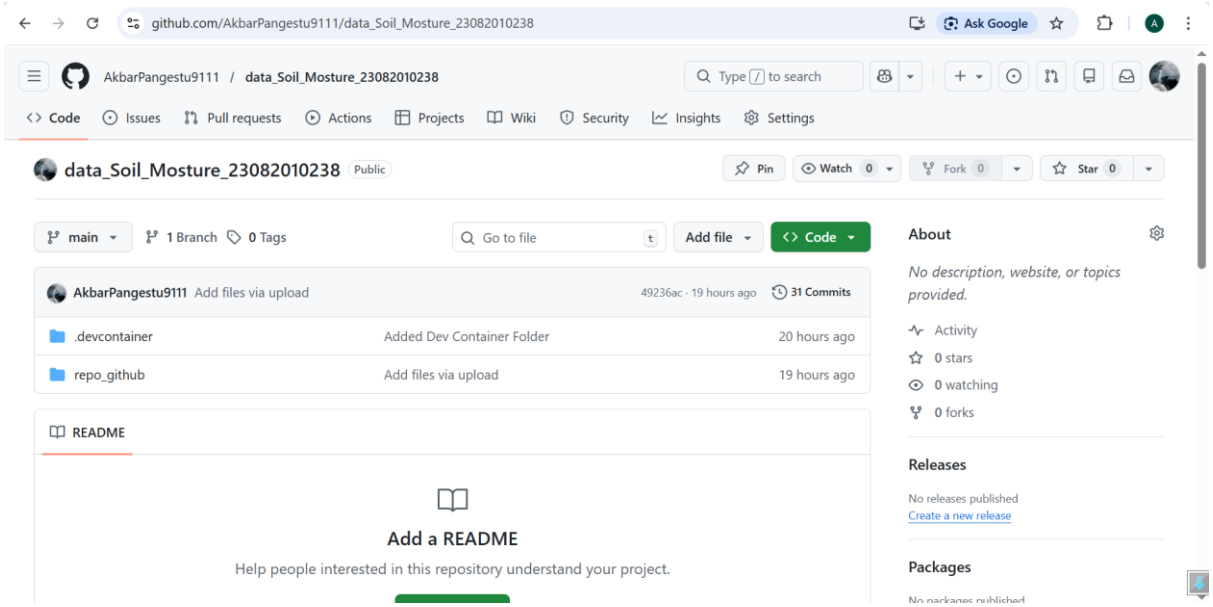
```
import os

folder_tujuan = r'C:\Users\user\.kaggle'

file_csv = [f for f in os.listdir(folder_tujuan) if f.endswith('.csv')]
print("File CSV yang ditemukan:")
for f in file_csv:
    print(f"- {f}")
```

```
File CSV yang ditemukan:
```

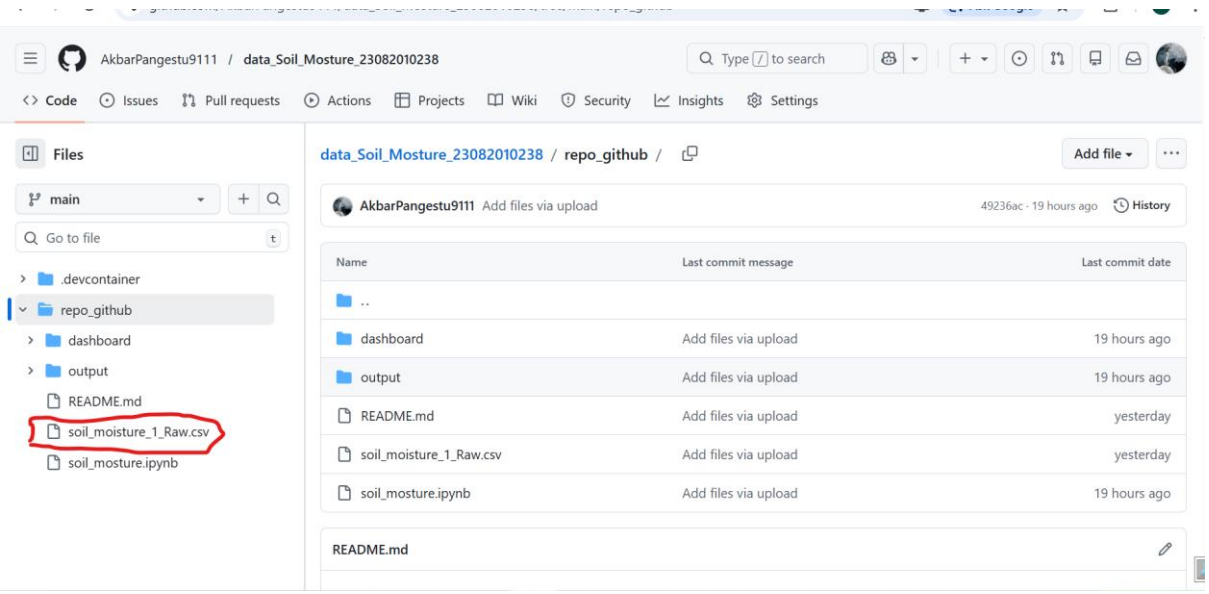
Penjelasan:

- folder_tujuan = r'C:\Users\user\.kaggle'
  Menentukan direktori tempat file ZIP diekstrak. Penggunaan r di depan string menjadikannya raw string, sehingga backslash dalam path Windows tidak diinterpretasikan sebagai escape character.
- file_csv = [f for f in os.listdir(folder_tujuan) if f.endswith('.csv')]
  Menggunakan list comprehension untuk membuat daftar semua file di dalam folder_tujuan yang memiliki ekstensi .csv. Fungsi os.listdir() mengembalikan daftar semua entri (file dan subdirektori) dalam direktori tersebut, dan kondisi f.endswith('.csv') memfilter hanya file CSV.
- print("File CSV yang ditemukan:")
  Menampilkan judul daftar.
- for f in file_csv:
  print(f"- {f}")
  Mencetak setiap nama file CSV dengan awalan tanda hubung, sehingga pengguna dapat melihat daftar file yang berhasil diekstrak.

C.Membuat repository github



D. Meletakan file dataset di repository



2.Procesing and Analysis

A.Eda lengkap:df.describe,df.isnull.sum

```
df = pd.read_csv(r'C:\Users\user\.kaggle\soil_moisture_1_Raw.csv')
print("Data berhasil dibaca!")
print(f"Shape: {df.shape}")
df.head()
print("\nInformasi Data:")
df.info()
print("\nStatistik Deskriptif:")
df.describe(include='all')
print("\nJumlah Missing Values per Kolom:")
print(df.isnull().sum())
```

```
Data berhasil dibaca!
Shape: (4117, 12)

Informasi Data:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4117 entries, 0 to 4116
Data columns (total 12 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   year       4117 non-null   int64
 1   month      4117 non-null   int64
 2   day        4117 non-null   int64
 3   hour       4117 non-null   int64
 4   minute     4117 non-null   int64
 5   second     4117 non-null   int64
 6   moisture0  4117 non-null   float64
 7   moisture1  4117 non-null   float64
 8   moisture2  4117 non-null   float64
 9   moisture3  4117 non-null   float64
 10  moisture4  4117 non-null   float64
 11  irrgation  4117 non-null   bool
dtypes: bool(1), float64(5), int64(6)
memory usage: 358.0 KB
```

```
Statistik Deskriptif:

Jumlah Missing Values per Kolom:
year         0
month        0
day          0
hour         0
minute       0
second       0
moisture0    0
moisture1    0
moisture2    0
moisture3    0
moisture4    0
irrgation    0
dtype: int64
```

Penjelasan :
Kode yang disusun bertujuan untuk mengunduh dataset dari Kaggle, mengekstraknya, dan melakukan eksplorasi data awal. Rangkaian perintah tersebut terdiri dari beberapa bagian:

- Persiapan lingkungan – Instalasi pustaka opendatasets dan pandas menggunakan pip. Langkah ini memastikan modul yang diperlukan tersedia, meskipun opendatasets tidak digunakan secara langsung dalam proses selanjutnya.
- Pengunduhan dataset – Perintah Kaggle API !kaggle datasets download -d amirmohammdjalili/soil-moisture-dataset mengunduh arsip ZIP dataset ke direktori kerja. Agar berhasil, diperlukan konfigurasi token Kaggle yang valid.
- Ekstraksi dan pembersihan – Modul zipfile digunakan untuk membuka dan mengekstrak seluruh isi file ZIP ke folder C:\Users\user\.kaggle. Setelah itu, file ZIP asli dihapus dengan os.remove untuk menghemat ruang.
- Verifikasi file CSV – Kode selanjutnya mendaftar semua file di folder tujuan dan menampilkan hanya yang berekstensi .csv. Hasilnya memastikan bahwa file soil_moisture_1_Raw.csv (dan kemungkinan file CSV lain) telah tersedia.
- Eksplorasi data awal (EDA) – Dataset dibaca ke dalam DataFrame pandas dengan pd.read_csv. Kemudian ditampilkan:
    - Ukuran data (shape) dan cuplikan lima baris pertama (head).
    - Informasi teknis (info) yang mencakup tipe data dan jumlah nilai non-null per kolom.
    - Statistik deskriptif (describe(include='all')) untuk melihat ringkasan numerik dan kategorikal.
    - Jumlah missing values per kolom menggunakan isnull().sum().

B.Analisis:Correlation heatmap, time series tren

a.Persiapan pembacaan data

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_style('whitegrid')
plt.rcParams['figure.figsize'] = (12, 6)

df = pd.read_csv(r'C:\Users\user\.kaggle\soil_moisture_1_Raw.csv')
print("Data mentah berhasil dibaca!")
print(f"Shape: {df.shape}")
print(df.head())
```

Penjelasan:

- Library yang diimpor mencakup pandas untuk manipulasi data, numpy untuk operasi numerik, serta matplotlib dan seaborn untuk visualisasi.
- Pengaturan gaya dan ukuran gambar dibuat agar tampilan grafik konsisten.
- Data mentah dibaca dan ditampilkan sekilas untuk memastikan tidak ada kesalahan.

b.Penanganan Outlier

```
df_clean = df.copy()

moisture_cols = ['moisture0', 'moisture1', 'moisture2', 'moisture3', 'moisture4']
Q1 = df_clean[moisture_cols].quantile(0.25)
Q3 = df_clean[moisture_cols].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = ((df_clean[moisture_cols] < lower_bound) | (df_clean[moisture_cols] > upper_bound)).any(axis=1)
print(f"\nJumlah outliers: {outliers.sum()} dari {len(df_clean)}")
df_clean = df_clean[~outliers].copy()
print(f"Shape setelah hapus outliers: {df_clean.shape}")

time_cols = ['year', 'month', 'day', 'hour', 'minute', 'second']
```

Penjelasan:

- Salinan data dibuat agar data asli tetap utuh.
- Metode IQR (Interquartile Range) digunakan untuk mendeteksi outlier pada kolom kelembaban. Nilai di luar rentang Q1-1.5IQR hingga Q3+1.5IQR dianggap outlier.
- Baris yang mengandung outlier dihapus, dan jumlahnya dilaporkan. Langkah ini bertujuan mengurangi gangguan analisis akibat nilai ekstrem.

c.Pembuatan indeks waktu

```
time_cols = ['year', 'month', 'day', 'hour', 'minute', 'second']
df_clean['datetime'] = pd.to_datetime(df_clean[time_cols])
df_clean.set_index('datetime', inplace=True)
df_clean.drop(columns=time_cols, inplace=True)
print("Kolom datetime berhasil dibuat sebagai index.")
```

Penjelasan:

- Kolom-kolom waktu yang terpisah digabung menjadi satu kolom datetime, kemudian dijadikan index. Hal ini memudahkan analisis deret waktu (time series).
- Kolom waktu asli dihapus karena sudah tidak diperlukan.

Hasil:

```
Data mentah berhasil dibaca!
Shape: (4117, 12)
   year  month  day  hour  minute  second  moisture0  moisture1  moisture2  \
0  2020      3    6    22      16      11       0.70       0.64       0.73
1  2020      3    6    22      17      11       0.70       0.64       0.71
2  2020      3    6    22      18      11       0.69       0.63       0.70
3  2020      3    6    22      19      11       0.69       0.63       0.70
4  2020      3    6    22      20      12       0.69       0.62       0.69

   moisture3  moisture4  irrgation
0       0.40       0.02      False
1       0.39       0.02      False
2       0.39       0.02      False
3       0.39       0.02      False
4       0.39       0.02      False

Jumlah outliers: 336 dari 4117
Shape setelah hapus outliers: (3781, 12)
```

```
Jumlah outliers: 336 dari 4117
Shape setelah hapus outliers: (3781, 12)
Kolom datetime berhasil dibuat sebagai index.
Kolom irigasi dikonversi ke numerik.

Data setelah cleaning:
                     moisture0  moisture1  moisture2  moisture3  moisture4  \
datetime
2020-03-06 22:16:11       0.70       0.64       0.73       0.40       0.02
2020-03-06 22:17:11       0.70       0.64       0.71       0.39       0.02
2020-03-06 22:18:11       0.69       0.63       0.70       0.39       0.02
2020-03-06 22:19:11       0.69       0.63       0.70       0.39       0.02
2020-03-06 22:20:12       0.69       0.62       0.69       0.39       0.02
```

3.Visualisasi data

A.Source code lengkap

```python
import os
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import streamlit as st


st.set_page_config(page_title="Dashboard Soil Moisture", layout="wide")
st.title("🌱 Dashboard Analisis Kelembaban Tanah (Soil Moisture)")


@st.cache_data
def load_data():
    """
    Membaca file CSV yang berada di folder yang sama dengan script ini.
    Menggunakan path absolut relatif terhadap lokasi script.
    """

    script_dir = os.path.dirname(os.path.abspath(__file__))

    file_name = 'soil_moisture_1_cleaned.csv'

    file_path = os.path.join(script_dir, file_name)

    if not os.path.exists(file_path):
        st.error(f"❌ File tidak ditemukan: {file_path}")
        st.stop()


    df = pd.read_csv(file_path, index_col=0, parse_dates=True)
```

```python


try:
    df = load_data()
    st.success("✅ Data berhasil dimuat!")
except Exception as e:
    st.error(f"Gagal memuat data: {e}")
    st.stop()


moisture_cols = [col for col in df.columns if 'moisture' in col]


st.sidebar.header("📋 Informasi Data")
st.sidebar.write(f"**Jumlah baris:** {df.shape[0]}")
st.sidebar.write(f"**Jumlah kolom:** {df.shape[1]}")
st.sidebar.write(f"**Rentang waktu:**")
st.sidebar.write(f"{df.index.min()} s/d {df.index.max()}")


st.header("📊 Statistik Deskriptif")
st.dataframe(df[moisture_cols].describe())


st.header("🔗 Korelasi Antar Sensor")
fig, ax = plt.subplots(figsize=(8, 6))
corr = df[moisture_cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5, ax=ax)
ax.set_title('Matriks Korelasi')
st.pyplot(fig)
```

```python
selected_sensors = st.multiselect(
    default=moisture_cols
)

if selected_sensors:
    fig, ax = plt.subplots(figsize=(12, 6))
    for col in selected_sensors:
        ax.plot(df.index, df[col], label=col, linewidth=1)
    ax.set_xlabel('Waktu')
    ax.set_ylabel('Nilai Kelembaban')
    ax.set_title('Time Series Semua Data')
    ax.legend()
    ax.grid(True)
    st.pyplot(fig)


    st.subheader("🔍 Filter Rentang Waktu")
    min_date = df.index.min().date()
    max_date = df.index.max().date()
    start_date = st.date_input("Tanggal mulai", min_date)
    end_date = st.date_input("Tanggal selesai", max_date)

    if start_date <= end_date:
        mask = (df.index.date >= start_date) & (df.index.date <= end_date)
        df_filtered = df.loc[mask]
        if not df_filtered.empty:
            fig2, ax2 = plt.subplots(figsize=(12, 4))
            for col in selected_sensors:
                ax2.plot(df_filtered.index, df_filtered[col], label=col, linewidth=1)
            ax2.set_xlabel('Waktu')
```

```python
axes_hist = axes_hist.flatten()
for i, col in enumerate(moisture_cols):
    axes_hist[i].hist(df[col], bins=30, edgecolor='black')
    axes_hist[i].set_title(col)

for j in range(len(moisture_cols), len(axes_hist)):
    fig_hist.delaxes(axes_hist[j])
fig_hist.suptitle('Histogram Nilai Kelembaban', fontsize=16)
plt.tight_layout()
st.pyplot(fig_hist)


st.subheader("Boxplot")
fig_box, ax_box = plt.subplots(figsize=(10, 6))
sns.boxplot(data=df[moisture_cols], ax=ax_box)
ax_box.set_title('Boxplot Sensor Kelembaban')
ax_box.set_ylabel('Nilai Kelembaban')
ax_box.grid(True)
st.pyplot(fig_box)


if st.checkbox("Tampilkan data mentah"):
    st.subheader("Data Cleaned")
    st.dataframe(df)


st.markdown("---")
st.markdown("Dashboard dibuat dengan ❤️ menggunakan Streamlit")
```

B.Penjelasan visualisasi

a.Statistik Deskriptif

```python
st.sidebar.write(f"{df.index.min()} s/d {df.index.max()}")


st.header("📊 Statistik Deskriptif")
st.dataframe(df[moisture_cols].describe())
```

- Menampilkan tabel statistik deskriptif (jumlah data, mean, standar deviasi, min, kuartil, max) untuk kelima sensor kelembaban.
- Tabel ini interaktif (dapat di-scroll) karena menggunakan st.dataframe.

b.Matrik korelasi antar sensor

```
st.header("🔗 Korelasi Antar Sensor")
fig, ax = plt.subplots(figsize=(8, 6))
corr = df[moisture_cols].corr()
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt='.2f', linewidths=0.5, ax=ax)
ax.set_title('Matriks Korelasi')
st.pyplot(fig)
```

- **Tujuan**: Menunjukkan seberapa erat hubungan linear antar sensor kelembaban.
- **Cara kerja**:
  - df[moisture_cols].corr() menghitung korelasi Pearson.
  - sns.heatmap memvisualisasikan matriks korelasi dengan warna. Nilai korelasi ditampilkan dalam kotak (annot=True).
  - Warna coolwarm memudahkan identifikasi korelasi positif (merah) dan negatif (biru).
- Grafik ini dibuat dengan matplotlib dan ditampilkan di halaman utama menggunakan st.pyplot(fig).

c.Tren kelembapan dari waktu ke waktu

Plot time series semua data:

```
if selected_sensors:
    fig, ax = plt.subplots(figsize=(12, 6))
    for col in selected_sensors:
        ax.plot(df.index, df[col], label=col, linewidth=1)
    ax.set_xlabel('Waktu')
    ax.set_ylabel('Nilai Kelembaban')
    ax.set_title('Time Series Semua Data')
    ax.legend()
    ax.grid(True)
    st.pyplot(fig)
```

Penejlasan:

- Untuk setiap sensor terpilih, digambar garis sepanjang waktu.
- Plot ini memberikan gambaran umum fluktuasi kelembaban.

Filter rentang waktu

```
st.subheader("🔍 Filter Rentang Waktu")
min_date = df.index.min().date()
max_date = df.index.max().date()
start_date = st.date_input("Tanggal mulai", min_date)
end_date = st.date_input("Tanggal selesai", max_date)

if start_date <= end_date:
    mask = (df.index.date >= start_date) & (df.index.date <= end_date)
    df_filtered = df.loc[mask]
    if not df_filtered.empty:
        fig2, ax2 = plt.subplots(figsize=(12, 4))
        for col in selected_sensors:
            ax2.plot(df_filtered.index, df_filtered[col], label=col, linewidth=1)
        ax2.set_xlabel('Waktu')
        ax2.set_ylabel('Kelembaban')
        ax2.set_title(f'Time Series ({start_date} s/d {end_date})')
        ax2.legend()
        ax2.grid(True)
        st.pyplot(fig2)
    else:
        st.warning("Tidak ada data pada rentang tanggal tersebut.")
else:
```

- Data difilter berdasarkan tanggal, lalu ditampilkan dalam plot baru yang lebih kecil. Ini memungkinkan pengamatan detail pada periode tertentu.
- Jika rentang tidak valid (tanggal akhir < tanggal mulai), muncul pesan error.

d.Distribusi kelembapan

Histogram

```
st.subheader("Histogram")
fig_hist, axes_hist = plt.subplots(nrows=3, ncols=2, figsize=(15, 10))
axes_hist = axes_hist.flatten()
for i, col in enumerate(moisture_cols):
    axes_hist[i].hist(df[col], bins=30, edgecolor='black')
    axes_hist[i].set_title(col)

for j in range(len(moisture_cols), len(axes_hist)):
    fig_hist.delaxes(axes_hist[j])
fig_hist.suptitle('Histogram Nilai Kelembaban', fontsize=16)
plt.tight_layout()
st.pyplot(fig_hist)
```

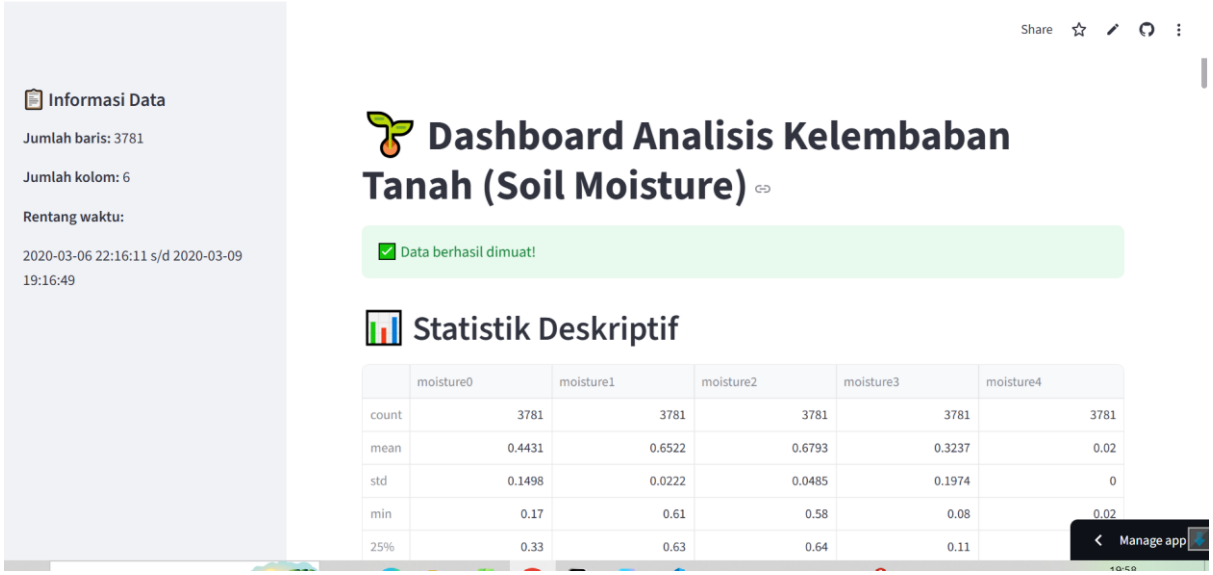
- Dibuat subplot 3 baris 2 kolom (total 6 area) untuk menampung 5 histogram sensor. Area ke-6 dihapus.
- Setiap histogram menunjukkan distribusi frekuensi nilai kelembaban satu sensor. Jumlah bin = 30.
- **Tujuan**: Melihat bentuk distribusi (apakah normal, menceng ke kiri/kanan, multimodal, dll.) dan mendeteksi adanya outlier.

Boxplot

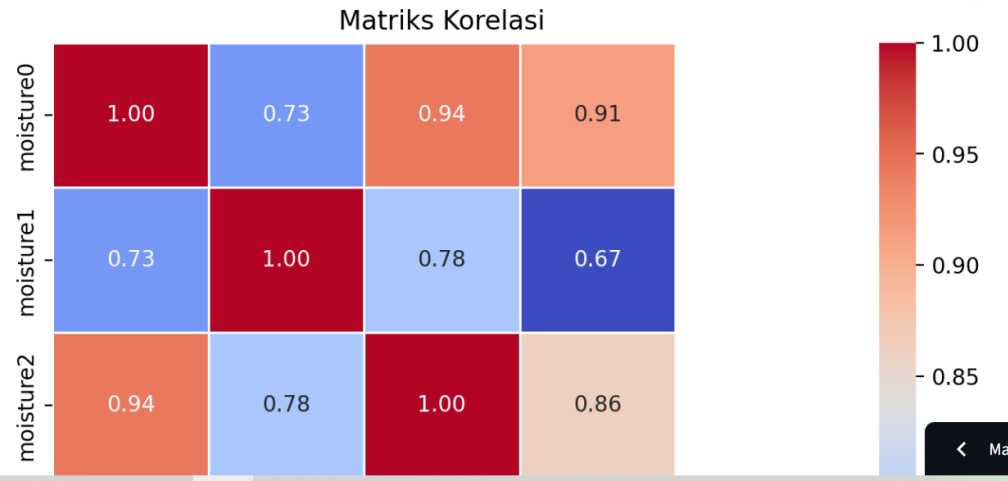
```
st.subheader("Boxplot")
fig_box, ax_box = plt.subplots(figsize=(10, 6))
sns.boxplot(data=df[moisture_cols], ax=ax_box)
ax_box.set_title('Boxplot Sensor Kelembaban')
ax_box.set_ylabel('Nilai Kelembaban')
ax_box.grid(True)
st.pyplot(fig_box)


if st.checkbox("Tampilkan data mentah"):
    st.subheader("Data Cleaned")
    st.dataframe(df)


st.markdown("---")
st.markdown("Dashboard dibuat dengan ❤️ menggunakan Streamlit")
```

- Boxplot menampilkan ringkasan lima angka (minimum, kuartil bawah, median, kuartil atas, maksimum) serta titik-titik yang dianggap outlier.
- **Tujuan**: Membandingkan sebaran antar sensor secara visual, melihat median, variabilitas, dan keberadaan outlier setelah proses pembersihan.

C.Dashboard



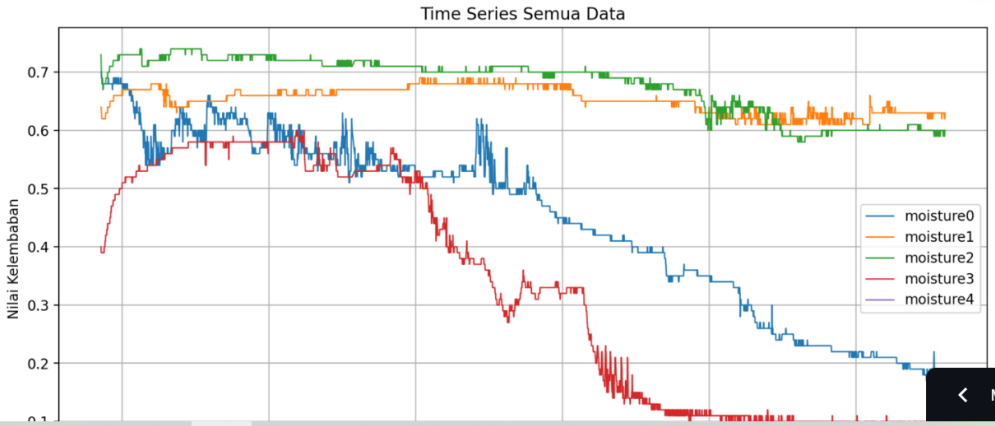
Korelasi Antar Sensor



Tren Kelembaban dari Waktu ke Waktu

Pilih sensor yang akan ditampilkan

moisture0 × moisture1 × moisture2 × moisture3 × moisture4 ×



D.Link Dashboard

https://datasoilmosture23082010238git-kpf2zsbyhs8y6eqhvdmt2x.streamlit.app/

3.Dokumentasi dan Governance

```
[2]: import pandas as pd
     from datetime import timedelta

     import pandas as pd
     from datetime import timedelta

     df = pd.read_csv(r'C:\Users\user\.kaggle\soil_moisture_1_Raw.csv', index_col=0, parse_dates=True)

     total = len(df)
     missing = df.isnull().sum().sum()
     accuracy = 1 - missing / total
     completeness = df.notnull().sum().sum() / total

     last_date = df.index.max()
     thirty_days_ago = last_date - timedelta(days=30)
     recent = df[df.index >= thirty_days_ago]
     timeliness = len(recent) / total

     print(f"Accuracy: {accuracy:.4f}")
     print(f"Completeness: {completeness:.4f}")
     print(f"Timeliness: {timeliness:.4f}")

     total = len(df)
     missing = df.isnull().sum().sum()
     accuracy = 1 - missing / total
     completeness = df.notnull().sum().sum() / total
```

```
total = len(df)
missing = df.isnull().sum().sum()
accuracy = 1 - missing / total
completeness = df.notnull().sum().sum() / total

last_date = df.index.max()
thirty_days_ago = last_date - timedelta(days=30)
recent = df[df.index >= thirty_days_ago]
timeliness = len(recent) / total

print(f"Accuracy: {accuracy:.4f}")
print(f"Completeness: {completeness:.4f}")
print(f"Timeliness: {timeliness:.4f}")
```

```
Accuracy: 1.0000
Completeness: 11.0000
Timeliness: 1.0000
Accuracy: 1.0000
Completeness: 11.0000
Timeliness: 1.0000
```

Intepretasi hasil:

Accuracy bernilai 1.0000, yang mengindikasikan tidak ditemukan nilai yang hilang (*missing values*) dalam dataset. Hal ini sesuai dengan definisi *Accuracy = 1 - (missing/total)*, di mana missing adalah jumlah sel kosong dan total adalah jumlah baris. Karena tidak ada sel kosong, maka akurasi mencapai nilai maksimum.

Completeness menunjukkan angka 11.0000. Angka ini perlu dipahami dalam konteks perhitungan yang digunakan. Dalam kode, *Completeness* dihitung sebagai (jumlah sel tidak null) / (jumlah baris). Karena dataset dibaca dengan parameter index_col=0, kolom pertama (misalnya year) dijadikan sebagai indeks, sehingga jumlah kolom data yang tersisa adalah 11. Seluruh sel pada 11 kolom tersebut terisi penuh (tidak ada null), sehingga total sel tidak null = 11 × jumlah baris. Dengan demikian, completeness = (11 × jumlah baris) / jumlah baris = 11. Angka ini merepresentasikan **rata-rata jumlah kolom yang terisi per baris**, bukan proporsi terhadap total sel. Jika dikehendaki proporsi, seharusnya penyebutnya adalah total seluruh sel (baris × kolom), yang akan menghasilkan nilai 1. Namun sesuai definisi yang diberikan dalam tugas, completeness dihitung dengan rumus tersebut, sehingga hasil 11.0000 menunjukkan bahwa setiap baris memiliki data lengkap pada 11 kolom.

Timeliness bernilai 1.0000, artinya seluruh data berada dalam rentang 30 hari terakhir dihitung dari tanggal terakhir dalam dataset. Hal ini bisa terjadi karena rentang waktu keseluruhan data memang kurang dari atau sama dengan 30 hari, sehingga semua baris termasuk dalam kategori "recent".